Linux Network Scripting with Lua

Lourival Vieira Neto, Victor Nogueira,
Ana Lúcia de Moura and Roberto Ierusalimschy

Ring-0 Networks and Departamento de Informática, PUC-Rio
Netdev 0x14

Kernel Scripting advocates that common OS kernels can be dynamically extended by using a high-level scripting language .

Lunatik is a Lua-based kernel-scripting framework for Linux

- Supports kernel developers for making subsystems scriptable
- Allows users to dynamically load and run scripts in the kernel

Why Lua?

- Small (250 KB) and fast scripting language
- Widely used for scripting network monitoring and security tools
  - e.g., Wireshark, Nmap, Snort

Sandboxing Kernel Scripts

Kernel scripts should not introduce malfunctioning to the system, e.g.:

- Crash or corrupt the system
- Run indefinitely or block an execution flow
- Corrupt resources owned by other users

Sandboxing Kernel Scripts

Lua provides dynamic memory management

- Scripts can't address memory directly
  - Only as Lua data types: strings, tables and userdata
- All data objects are allocated internally by the Lua VM
  - Garbage Collector
- Custom memory allocator
  - Cap memory allocation

Sandboxing Kernel Scripts

ring-0

Lua provides fully isolated execution states

- Lua states are created empty
  - Only language operators are initially available
  - Must explicitly load libraries into the state
    - e.g., Luadata, LuaRCU

Sandboxing Kernel Scripts

Lua provides a single-threaded execution environment

- Lua doesn't provide primitives for synchronization mutexes
  - Scripts cannot explicitly lock a kernel flow
  - Only collaborative multitasking (coroutines)
- Lua can interrupt a script after running a specific amount of instructions
  - Cap instructions executed (lua_sethook)
- Lunatik permits the creation of multiple Lua execution states inside the kernel for multitasking

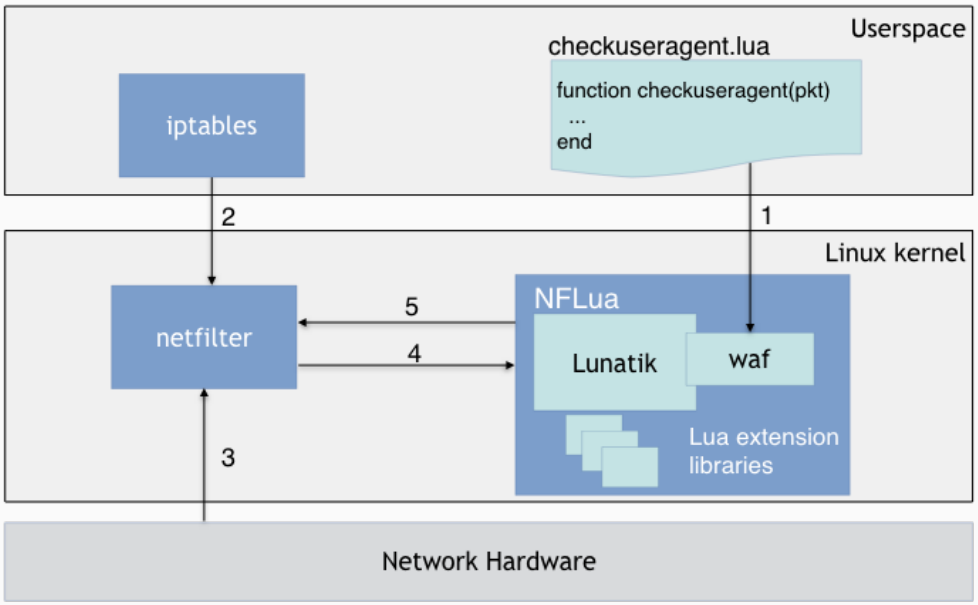
Sandboxing Kernel Scripts

NFLua and XDPLua allow only network administrators

- Uses Netlink socket
- Checks CAP_NET_ADMIN capability on every access

Netfilter extension for packet filtering using Lua

- Layer 7 filtering
- Widely used by network operators for advanced security and network monitoring
- Loadable kernel module

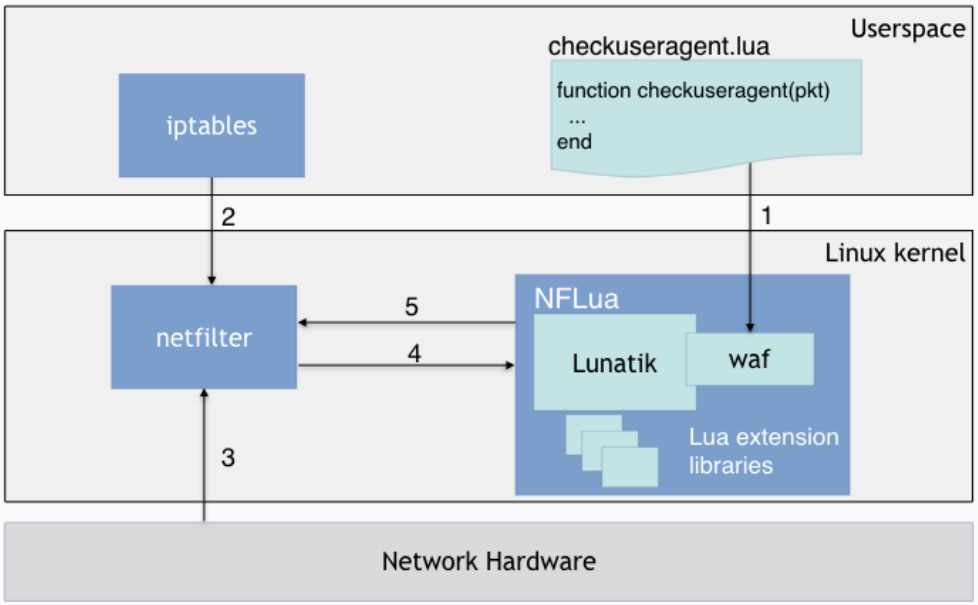


1. User creates a Lua state (waf) and loads a function (checkuseragent)

```
1    nfluactl create waf
2    nfluactl execute checkuseragent.lua
```

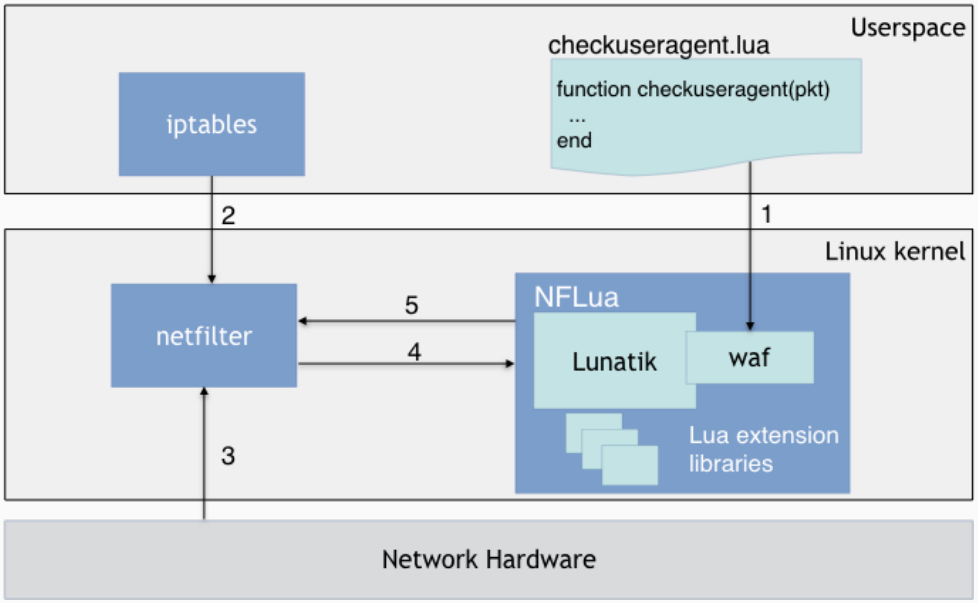
```
1  function checkuseragent(pkt)
2    -- extracts User-Agent HTTP header
3    local pattern = "User%-Agent:%s(.-)\r\n"
4    local useragent = string.match(pkt, pattern)
5
6    return blocklist[useragent]
7  end
```

Figure: Inspecting the *User-Agent* HTTP header in Lua



2. User sets an iptables rule to send packets destined to TCP port 80 to NFLua

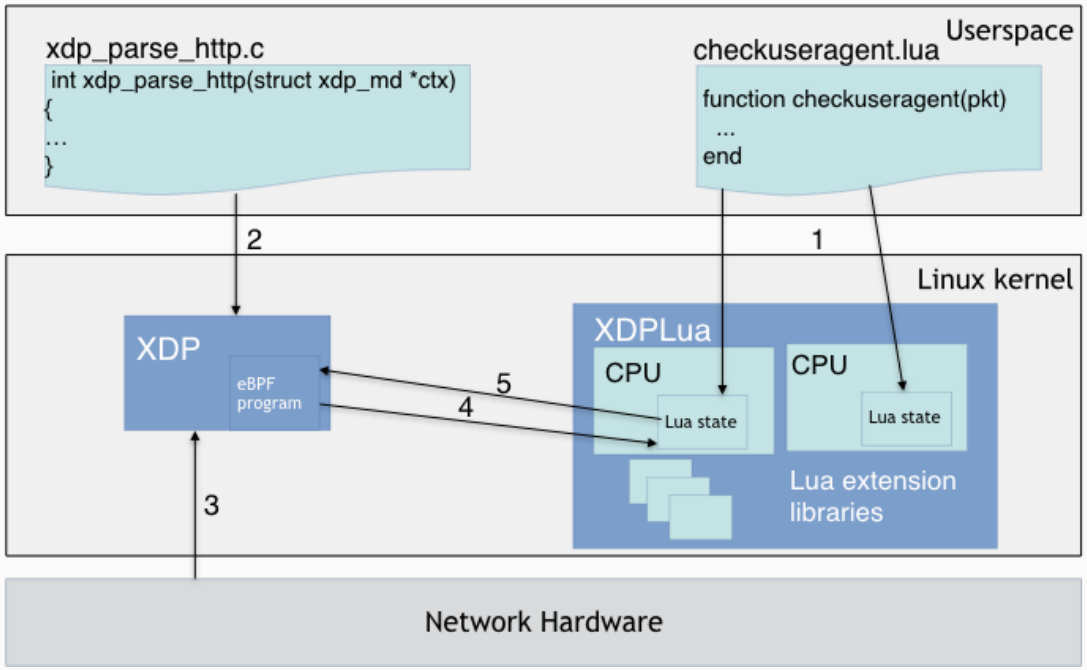
```
1   iptables -A INPUT -m TCP --dport 80 -m lua
2     --state waf --function checkuseragent -j REJECT
```



3. Netfilter receives a packet on TCP port 80
4. Netfilter sends the packet to NFLua; then NFLua calls the function checkuseragent, running in the state waf
5. If a match is found for the packet, Netfilter is instructed to terminate the connection

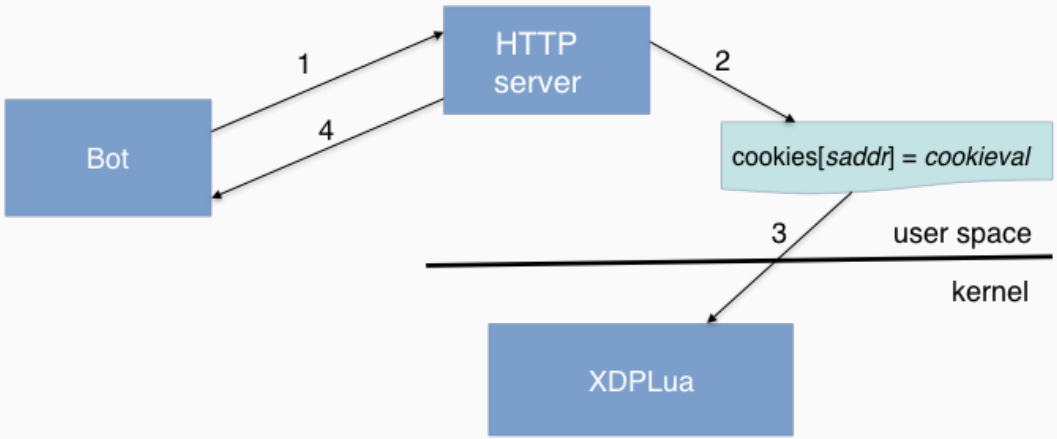
XDPLua

- Evolution of NFLua
- Extends XDP to use Lua
- One Lua state per CPU
- Adds expressiveness and dynamism
- eBPF scripting with Lua
    - Wrappers to the Lua C API as eBPF helpers



1. User loads Lua function (checkuseragent) into XDPLua
2. User loads the eBPF program into XDP
3. Packet arrives at XDP
4. eBPF program calls the Lua function (checkuseragent)
5. Lua function returns its verdict

JavaScript Challenge



1. Client sends HTTP request
2. Server generates random cookie
3. Cookie is loaded into XDPLua
4. Server sends HTTP response with embedded JavaScript

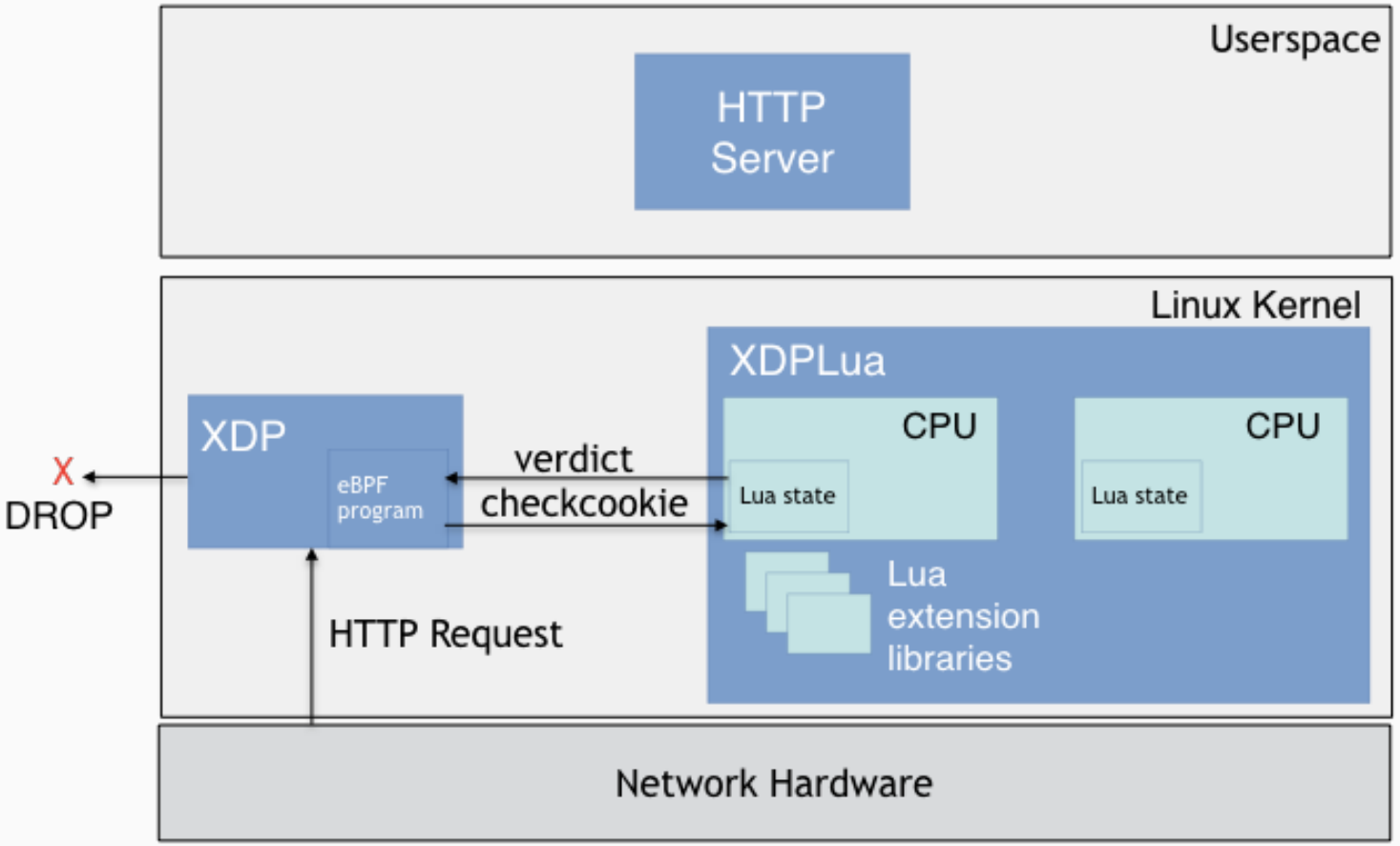
```lua
function checkcookie(pkt, saddr)
  -- checks if challenge is not set yet (first request)
  if not cookies[saddr] then
    return true
  end

  -- extracts __xdp cookie
  local pattern = "Cookie:%s*=__xdp=(%d+)%s*"
  local cookieval = string.match(tostring(pkt), pattern)

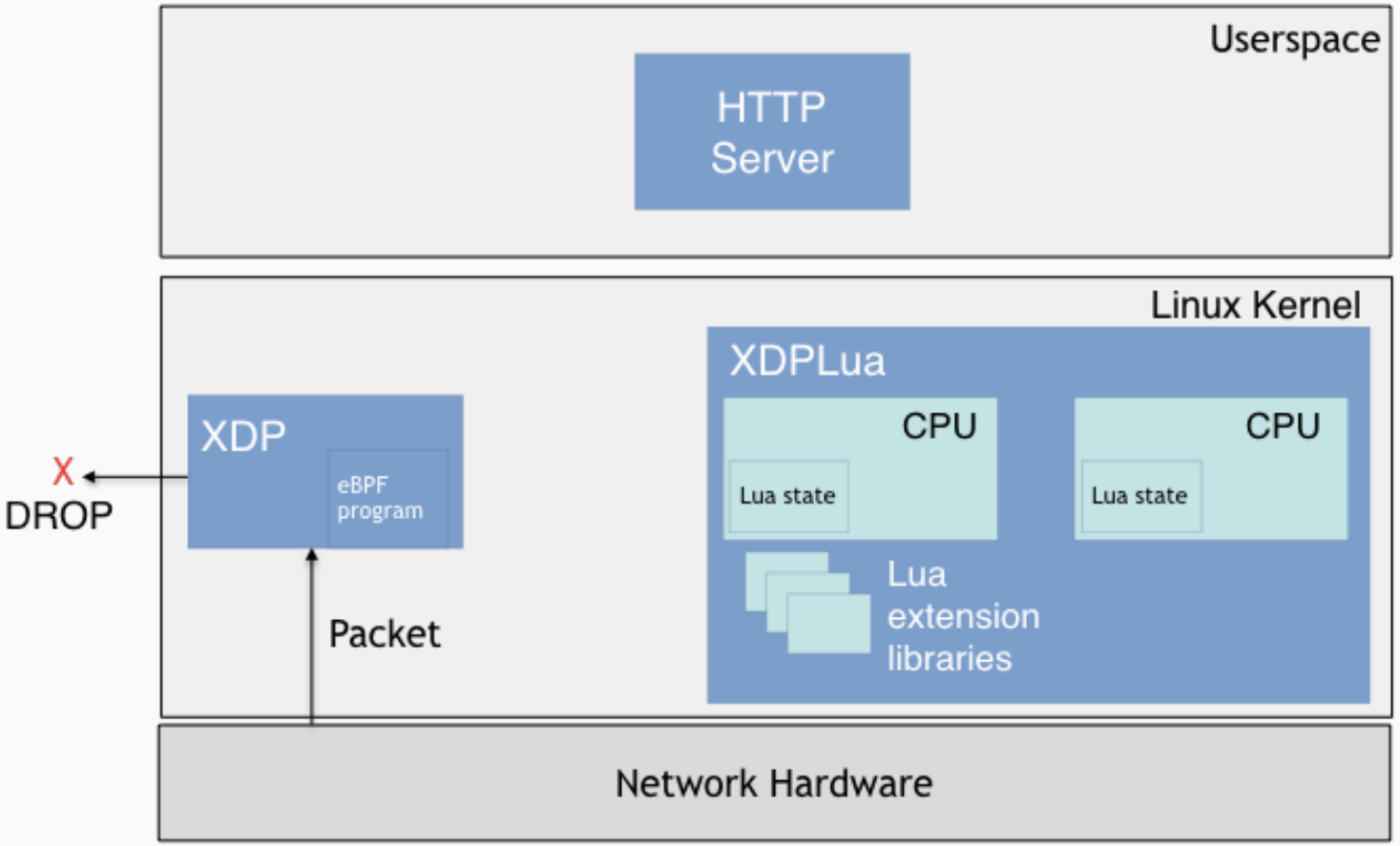
  -- checks cookie's value
  return cookies[saddr] == cookieval
end
```

Figure: JavaScript Challenge's Lua function

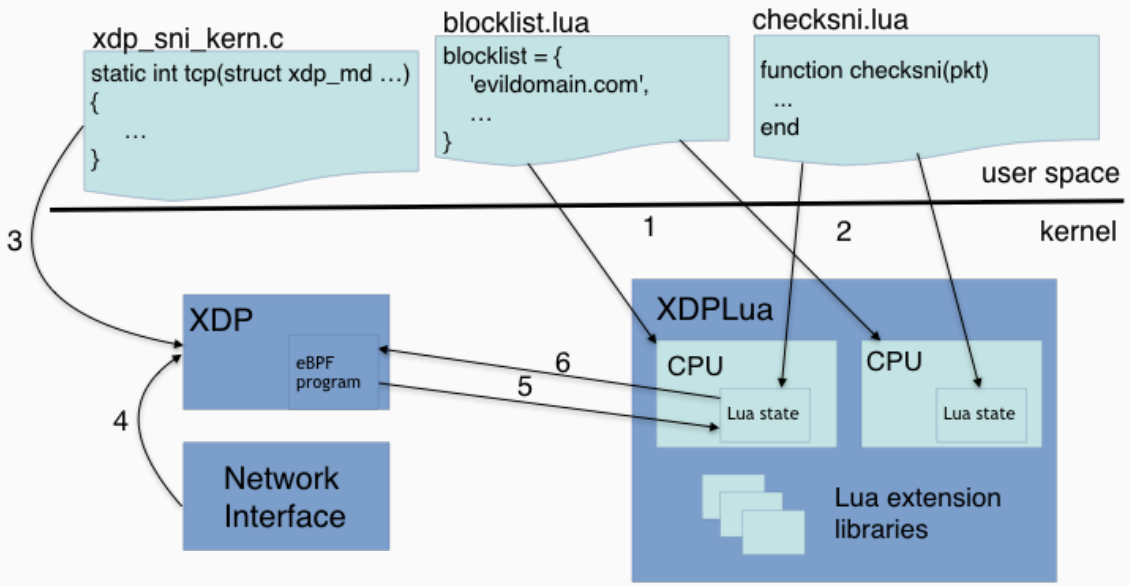
JavaScript Challenge



JavaScript Challenge



Access Control

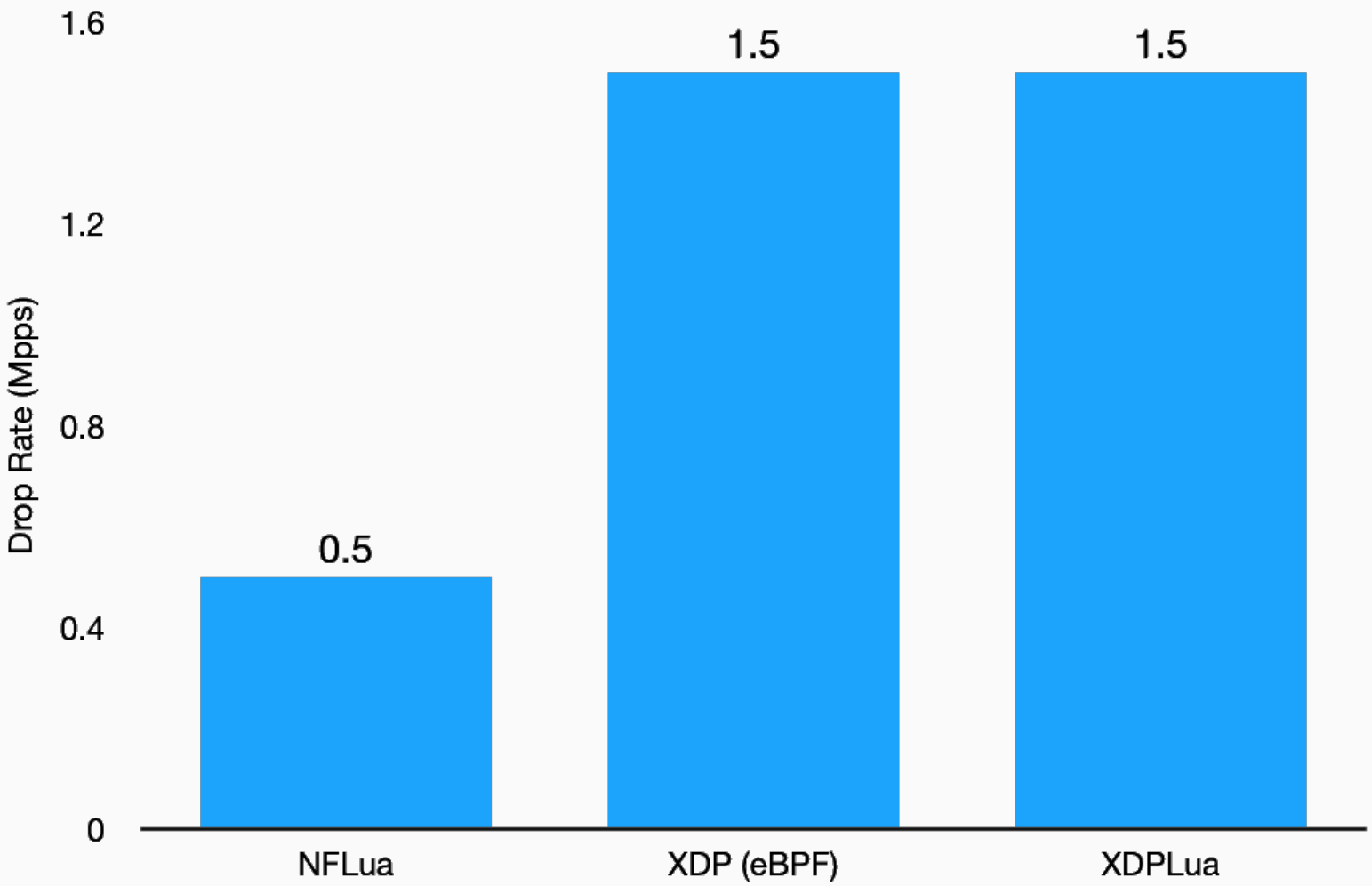


1. User loads blocklist into XDPLua
2. User loads Lua function (checksni) into XDPLua
3. User loads the eBPF program into a XDP
4. Packet arrives at XDP
5. The eBPF program calls the Lua function (checksni)
6. The Lua function extracts the SNI, verifies if it's in the blocklist, and returns its verdict

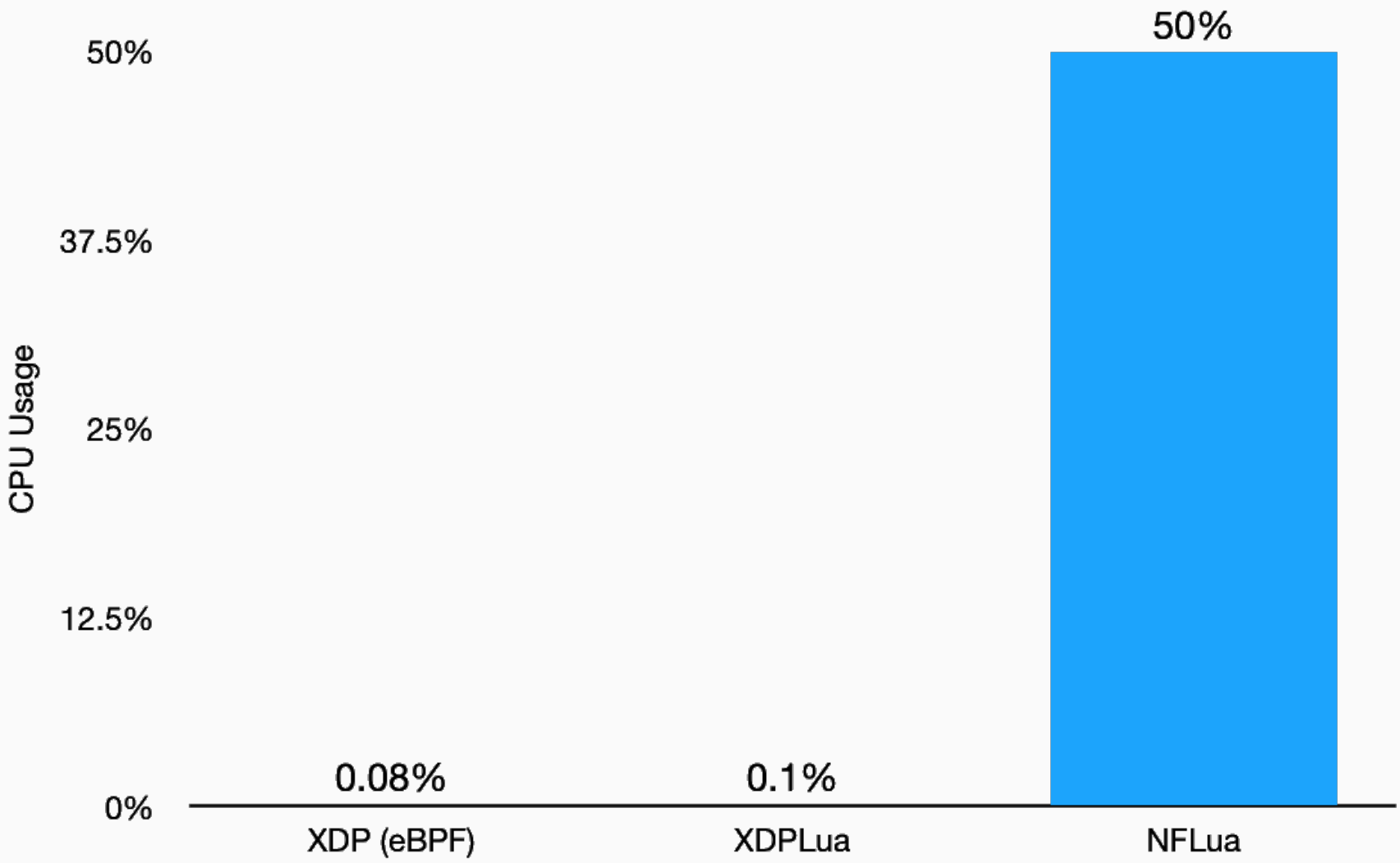
Benchmarks

- Implemented Access Control for XDP (eBPF), XDPLua and NFLua
  - XDPLua and NFLua shares the same Lua script implementation
- Used Trafgen on the client to send *TLS client hello* packets
- Measured *drop rate* and CPU usage on the server
- Fully virtualized environment
  - CPU with 8 cores and running at 3.00 GHz, 32 GB of RAM
  - 10 Gbps Virtio Network Interface

Results



Results



Final Remarks

ring-0

- NFLua is currently present in around 20 million home routers to protect and monitor over 500 million devices
  - Used by network operators for implementing advanced cybersecurity and network monitoring
- XDPLua is the evolution of NFLua, applying lessons learnt from NFLua's development
- XDPLua was designed to use Lua cooperatively with eBPF
  - Expressiveness and ease of use of Lua
  - Performance of eBPF
- XDPLua is currently used in the *Ring-0 Firewall* running in PoPs with 10 Gbps bandwidth
  - Peaks of around 5.3 Gbps , only up to 4% of CPU
- XDP doesn't support extensions as LKM
  - An out-of-tree binding is considerably harder to implement and maintain
- We apply sandboxing techniques
  - Instead of limiting the language or imposing an in-kernel verifier

```lua
local contacts = {
  "lourival.neto@ring-0.io",
  "victor.nogueira@ring-0.io"
}

-- Questions?
if further then
  for _, c in ipairs(contacts) do
    mailto(c)
  end
end

print("Thank You!")
```